



Exploración de datos con Pandas

**Clase 3 - Métodos básicos de exploración de datos con
Pandas**





¿Qué aprenderás en esta sesión?

Aprenderemos a utilizar los métodos `head()`, `tail()`, `info()` y `describe()` de la biblioteca `pandas`, fundamentales para realizar una primera exploración técnica de un `DataFrame`. Estos métodos permiten conocer rápidamente la estructura general, el tipo de variables, la cantidad de datos disponibles y una primera aproximación estadística a los valores contenidos.

Desafío inicial

Imagina que trabajas en el área de análisis de una empresa de transportes interurbanos. Acaban de entregarte un archivo con más de 10.000 registros sobre viajes realizados: fechas, ciudades, precios de pasajes, número de pasajeros, tipo de bus, etc. Tienes menos de 15 minutos para dar una visión preliminar del contenido del archivo: ¿tiene datos completos?, ¿cuántos registros hay?, ¿cómo se ven los primeros datos?, ¿qué tipo de variables contiene?

¿Cómo podrías responder todas estas preguntas sin revisar manualmente las 10.000 filas?

Para resolverlo de forma eficiente, necesitas dominar los métodos de exploración rápida que ofrece pandas. Ellos te permitirán extraer insights clave sobre el contenido y la calidad de los datos, en pocos segundos.



/* Exploración inicial con head() y tail()
***/**

Exploración inicial con head() y tail()

¿Cómo podemos visualizar rápidamente los primeros y últimos registros de un conjunto de datos sin cargarlo completo?

Cuando trabajamos con datasets extensos, visualizar todos los registros no solo es ineficiente, sino que puede colapsar la interfaz o generar ruido visual. Por eso, pandas incorpora funciones para examinar una muestra estructurada del inicio y del final del DataFrame.

1

head(n=5)

Muestra las primeras n filas del DataFrame. Si no se especifica n, por defecto devuelve 5 registros.

```
df.head()    # Primeras 5 filas  
df.head(10)  # Primeras 10 filas
```

Usos comunes:

- Verificar formato de columnas.
- Comprobar que la carga del archivo fue exitosa.
- Detectar encabezados mal cargados (cuando los datos empiezan en la fila 2, por ejemplo).

tail(n=5)

Muestra las últimas n filas del DataFrame.

```
df.tail()    # Últimas 5 filas  
df.tail(3)   # Últimas 3 filas
```

Usos comunes:

- Verificar si hay datos nulos al final.
- Revisar registros más recientes (en tablas ordenadas por fecha).
- Detectar errores de formato en las filas finales.

Ejemplo aplicado:

```
import pandas as pd  
df = pd.read_csv('viajes_enero.csv')  
print(df.head())  
print(df.tail(2))
```

Consideraciones importantes

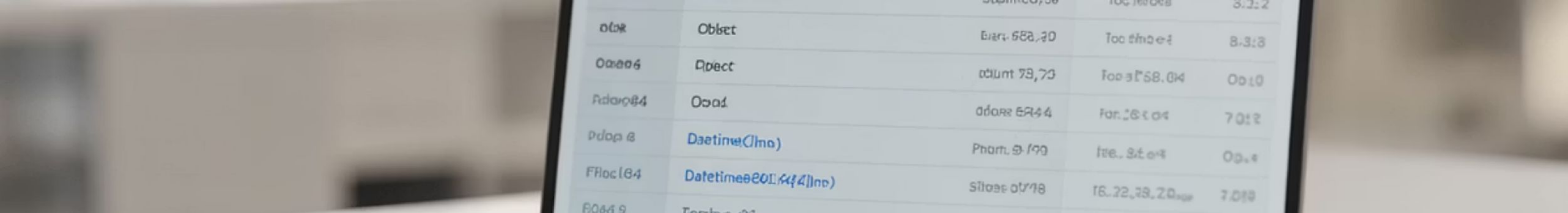
❗ ¡Importante! Estos métodos no modifican el DataFrame; solo muestran una vista parcial de forma segura.

Ahora que sabemos cómo revisar rápidamente los primeros y últimos registros de una tabla con `head()` y `tail()`, podemos hacernos una pregunta clave:

¿Cómo saber cuántos datos hay realmente, qué columnas existen y qué tipo de información contiene cada una?

Esa respuesta nos la ofrece el método `info()`, que exploraremos a continuación.





00000	Object	555,55	100,00000	8.313
00001	Object	555,55	100,00000	8.313
00002	Object	555,55	100,00000	8.313
00003	Object	555,55	100,00000	8.313
00004	Object	555,55	100,00000	8.313
00005	Object	555,55	100,00000	8.313
00006	Object	555,55	100,00000	8.313
00007	Object	555,55	100,00000	8.313
00008	Object	555,55	100,00000	8.313
00009	Object	555,55	100,00000	8.313
00010	Object	555,55	100,00000	8.313
00011	Object	555,55	100,00000	8.313
00012	Object	555,55	100,00000	8.313
00013	Object	555,55	100,00000	8.313
00014	Object	555,55	100,00000	8.313
00015	Object	555,55	100,00000	8.313
00016	Object	555,55	100,00000	8.313
00017	Object	555,55	100,00000	8.313
00018	Object	555,55	100,00000	8.313
00019	Object	555,55	100,00000	8.313
00020	Object	555,55	100,00000	8.313
00021	Object	555,55	100,00000	8.313
00022	Object	555,55	100,00000	8.313
00023	Object	555,55	100,00000	8.313
00024	Object	555,55	100,00000	8.313
00025	Object	555,55	100,00000	8.313
00026	Object	555,55	100,00000	8.313
00027	Object	555,55	100,00000	8.313
00028	Object	555,55	100,00000	8.313
00029	Object	555,55	100,00000	8.313
00030	Object	555,55	100,00000	8.313
00031	Object	555,55	100,00000	8.313
00032	Object	555,55	100,00000	8.313
00033	Object	555,55	100,00000	8.313
00034	Object	555,55	100,00000	8.313
00035	Object	555,55	100,00000	8.313
00036	Object	555,55	100,00000	8.313
00037	Object	555,55	100,00000	8.313
00038	Object	555,55	100,00000	8.313
00039	Object	555,55	100,00000	8.313
00040	Object	555,55	100,00000	8.313
00041	Object	555,55	100,00000	8.313
00042	Object	555,55	100,00000	8.313
00043	Object	555,55	100,00000	8.313
00044	Object	555,55	100,00000	8.313
00045	Object	555,55	100,00000	8.313
00046	Object	555,55	100,00000	8.313
00047	Object	555,55	100,00000	8.313
00048	Object	555,55	100,00000	8.313
00049	Object	555,55	100,00000	8.313
00050	Object	555,55	100,00000	8.313
00051	Object	555,55	100,00000	8.313
00052	Object	555,55	100,00000	8.313
00053	Object	555,55	100,00000	8.313
00054	Object	555,55	100,00000	8.313
00055	Object	555,55	100,00000	8.313
00056	Object	555,55	100,00000	8.313
00057	Object	555,55	100,00000	8.313
00058	Object	555,55	100,00000	8.313
00059	Object	555,55	100,00000	8.313
00060	Object	555,55	100,00000	8.313
00061	Object	555,55	100,00000	8.313
00062	Object	555,55	100,00000	8.313
00063	Object	555,55	100,00000	8.313
00064	Object	555,55	100,00000	8.313
00065	Object	555,55	100,00000	8.313
00066	Object	555,55	100,00000	8.313
00067	Object	555,55	100,00000	8.313
00068	Object	555,55	100,00000	8.313
00069	Object	555,55	100,00000	8.313
00070	Object	555,55	100,00000	8.313
00071	Object	555,55	100,00000	8.313
00072	Object	555,55	100,00000	8.313
00073	Object	555,55	100,00000	8.313
00074	Object	555,55	100,00000	8.313
00075	Object	555,55	100,00000	8.313
00076	Object	555,55	100,00000	8.313
00077	Object	555,55	100,00000	8.313
00078	Object	555,55	100,00000	8.313
00079	Object	555,55	100,00000	8.313
00080	Object	555,55	100,00000	8.313
00081	Object	555,55	100,00000	8.313
00082	Object	555,55	100,00000	8.313
00083	Object	555,55	100,00000	8.313
00084	Object	555,55	100,00000	8.313
00085	Object	555,55	100,00000	8.313
00086	Object	555,55	100,00000	8.313
00087	Object	555,55	100,00000	8.313
00088	Object	555,55	100,00000	8.313
00089	Object	555,55	100,00000	8.313
00090	Object	555,55	100,00000	8.313
00091	Object	555,55	100,00000	8.313
00092	Object	555,55	100,00000	8.313
00093	Object	555,55	100,00000	8.313
00094	Object	555,55	100,00000	8.313
00095	Object	555,55	100,00000	8.313
00096	Object	555,55	100,00000	8.313
00097	Object	555,55	100,00000	8.313
00098	Object	555,55	100,00000	8.313
00099	Object	555,55	100,00000	8.313

Revisión estructural con info()

¿Cómo podemos conocer cuántos datos hay, qué tipo de columnas existen y si hay valores faltantes?

El método .info() entrega una visión estructural y compacta del DataFrame, útil para diagnóstico de calidad de datos y planificación de limpieza.

```
df.info()
```

Este método nos indica:

- Cantidad total de registros (filas).
- Cantidad de columnas.
- Nombres de cada columna.
- Cantidad de valores no nulos por columna.
- Tipo de dato de cada columna (int64, float64, object, datetime64, etc.).
- Memoria utilizada por el DataFrame.

¿Por qué es importante?

- Permite detectar columnas con valores faltantes (si hay menos de n datos en una columna).
- Ayuda a decidir si es necesario convertir tipos de datos (por ejemplo, pasar un object a datetime).
- Permite estimar impacto en memoria para grandes volúmenes de datos.

Ejemplo aplicado:

```
df = pd.read_csv('ventas_sucursales.csv')  
df.info()
```

Pandas Datframe Info Method

```
In[100]: df.info()  
Out[100]: <class 'pandas.core.frame.DataFrame'>  
Int64Index: 10 entries, 0 to 9  
Data columns (total 4 columns):  
#   Column             Non-Null Count  Dtype  Dtype N  
#   :                 :              :   >   >   :  
0   id                 10 non-null    int64  int64  0  
1   sucursal           10 non-null    object  object  0  
2   fecha              10 non-null    object  object  0  
3   monto              10 non-null    float64 float64  0  
dtypes: int64(1), object(3), float64(1)  
memory usage: 1.1 KB
```

	Column	Non-Null Count	Dtype	Dtype	N
0	id	10 non-null	int64	int64	0
1	sucursal	10 non-null	object	object	0
2	fecha	10 non-null	object	object	0
3	monto	10 non-null	float64	float64	0

/* Estrategia de exploración integrada */

Preparación para análisis predictivo

Ten presente que, si estás preparando un modelo predictivo, `.info()` es la primera herramienta para identificar qué columnas necesitas limpiar o transformar.

Luego de identificar la cantidad de filas y columnas, los nombres de las variables y los tipos de datos con `info()`, lo siguiente es preguntarnos:

¿Cómo están distribuidos los valores dentro de las columnas? ¿Hay valores extremos? ¿Cuál es la media, la mediana o el rango de los datos numéricos?

Para responder estas preguntas, pandas nos ofrece el método `describe()`, que veremos ahora en profundidad.



Análisis estadístico preliminar con describe()

¿Cómo podemos obtener un resumen estadístico de todas las columnas numéricas de forma inmediata?

El método `.describe()` calcula automáticamente estadísticas descriptivas de todas las columnas numéricas (por defecto), entregando una síntesis rápida y poderosa del contenido.

```
df.describe()
```

Muestra:

- `count`: cantidad de datos no nulos.
- `mean`: media aritmética.
- `std`: desviación estándar.
- `min`: valor mínimo.
- `25%`: primer cuartil.
- `50%`: mediana.
- `75%`: tercer cuartil.
- `max`: valor máximo.

Usos frecuentes:

- Detectar valores extremos o atípicos.
- Observar asimetrías (por ejemplo, si media y mediana difieren mucho).
- Obtener métricas rápidas para reportes.

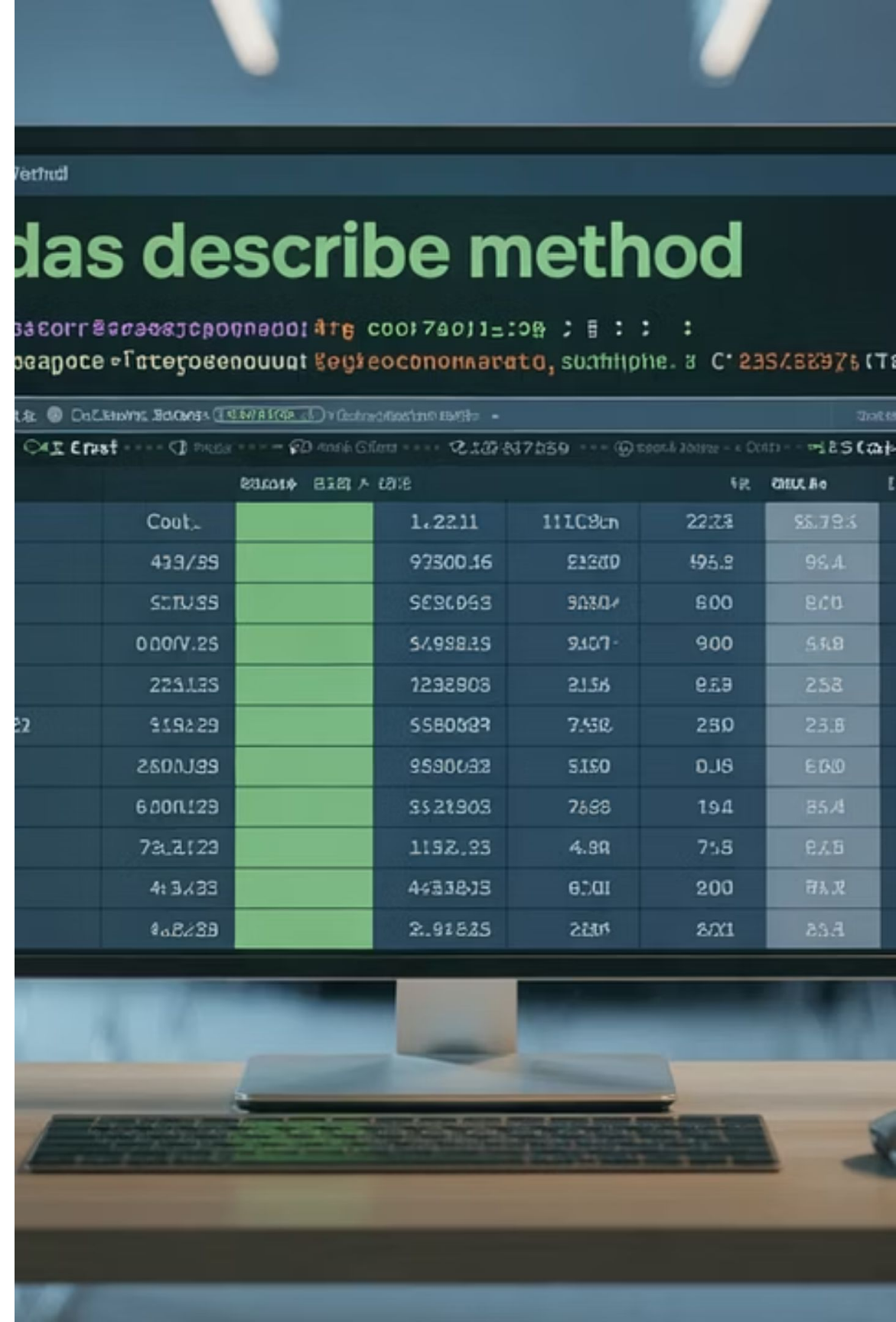


Ejemplo aplicado de describe()

```
df = pd.read_csv('pasajeros.csv')
df.describe()
```

 **Para explorar columnas no numéricas:**

```
df.describe(include='object')  
# para texto  
df.describe(include='all')  
# para todo tipo de datos
```



/* Preparación para análisis predictivo */

Estrategia de exploración integrada

Ya entendemos cómo obtener estadísticas básicas con `describe()` y cómo interpretar su salida. Sin embargo, en un entorno laboral real no usamos estos métodos por separado, sino en conjunto, como parte de una estrategia de exploración rápida.

Entonces, ¿cómo organizamos estos métodos para realizar un diagnóstico inicial eficiente cada vez que cargamos un nuevo dataset?

En el siguiente bloque abordaremos buenas prácticas para combinar estas funciones de forma ordenada y significativa.



Combinando métodos para diagnóstico inicial

¿Cómo planificamos una exploración rápida, completa y ordenada de un nuevo DataFrame?

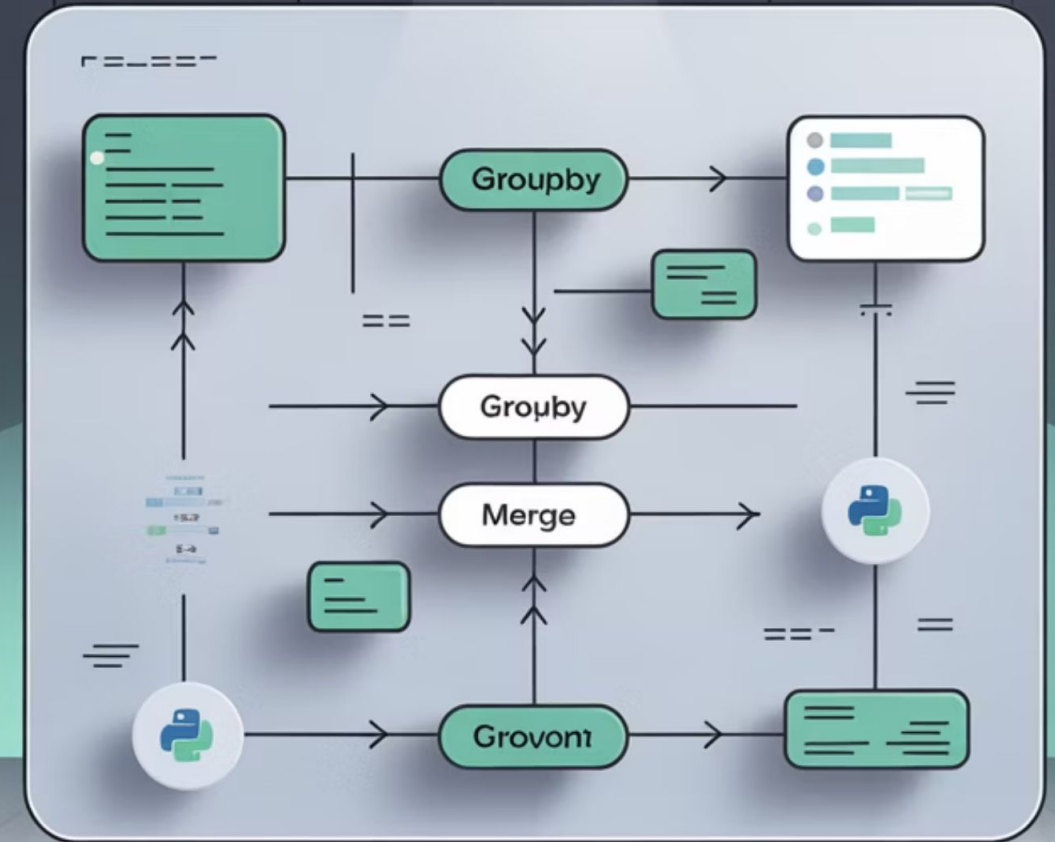
Una buena práctica profesional es crear una rutina de revisión básica que combine estos métodos. Por ejemplo:

```
print("Vista inicial:")
print(df.head())
print("\nResumen de estructura:")
print(df.info())
print("\nEstadísticas básicas:")
print(df.describe())
```

Esta combinación:

- Confirma si los datos se cargaron correctamente.
- Permite estimar qué columnas están limpias, cuáles necesitan tratamiento.
- Aporta una visión técnica inmediata para comunicar hallazgos o alertas.

las Methods



Ejercicio - Exploración rápida de registros de venta

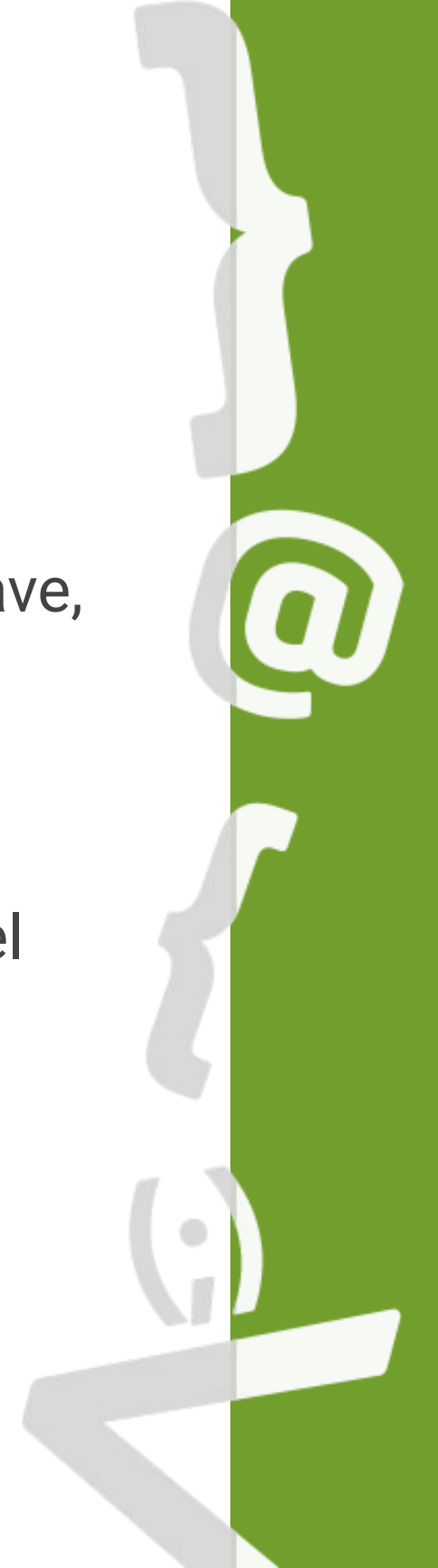


Ejercicio

Instrucciones

Objetivo: Aplicar métodos de exploración rápida (head, tail, info, describe) a un DataFrame, para analizar su estructura general, tipos de datos y estadísticas clave, replicando el diagnóstico inicial en un flujo de trabajo real.

Situación: Trabajas en el equipo de análisis de una cadena de farmacias. Has recibido un archivo CSV con los registros de ventas de medicamentos durante el último mes. Antes de hacer cualquier cálculo, debes entregar a tu jefatura un resumen: ¿cuántos registros hay?, ¿qué variables contiene el archivo?, ¿hay columnas con valores faltantes?, ¿cómo se distribuyen los precios?



Ejercicio

Instrucciones

Crea el siguiente DataFrame (simula datos de ventas):

```
import pandas as pd
datos = {'Producto': ['Paracetamol', 'Ibuprofeno', 'Amoxicilina', 'Aspirina', 'Loratadina'],
        'Precio': [1200, 1500, 2500, 900, 1800],
        'Stock': [35, 20, 0, 50, 40],
        'Farmacia': ['Central', 'Norte', 'Sur', 'Central', 'Oriente']}
df = pd.DataFrame(datos)
```

Ejercicio

Instrucciones

Ejecuta los siguientes métodos y analiza:

```
print(df.head())  
print(df.tail())  
print(df.info())  
print(df.describe())
```

1

Comparte tus respuestas a:

- ¿Qué tipo de variables contiene el DataFrame?
- ¿Hay alguna columna con valores faltantes?
- ¿Cuál es el producto más caro? ¿Y el más barato?
- ¿Cómo es la distribución de los valores de stock?



Ejercicio - Diagnóstico estructural y estadístico



Ejercicio

Instrucciones

Objetivo: Utilizar los métodos head, tail, info y describe para explorar un conjunto de datos simulado y elaborar una breve interpretación del estado y contenido del archivo.

Situación: Eres responsable de validar la calidad de un archivo de asistencia a capacitaciones. Contiene registros por persona, fecha, duración, modalidad y región. Debes explorar el archivo con pandas y anotar tus observaciones preliminares.



Ejercicio

Instrucciones

1. Crea el siguiente DataFrame:

```
data = {'Nombre': ['Luis', 'Andrea', 'Camila', 'Jorge',  
'María'],  
        'Duración (hrs)': [2, 3, 4, 2, 3],  
        'Modalidad': ['Online', 'Presencial', 'Online', 'Presencial',  
        'Online'],  
        'Región': ['RM', 'Biobío', 'RM', 'Valparaíso', 'Araucanía']}  
df = pd.DataFrame(data)
```



Ejercicio

Instrucciones

1. Aplica los métodos:

```
df.head()  
df.tail()  
df.info()  
df.describe()
```

2. Reflexiona:

☐

¿Qué variables son categóricas?

☐

¿Hay valores faltantes?

☐

¿Cuál es la duración promedio de las sesiones?

☐

¿Cuál fue la modalidad más frecuente?

Resumen



Exploración

Exploramos los métodos `head()`, `tail()`, `info()` y `describe()` como herramientas esenciales para el diagnóstico rápido de datos en pandas.

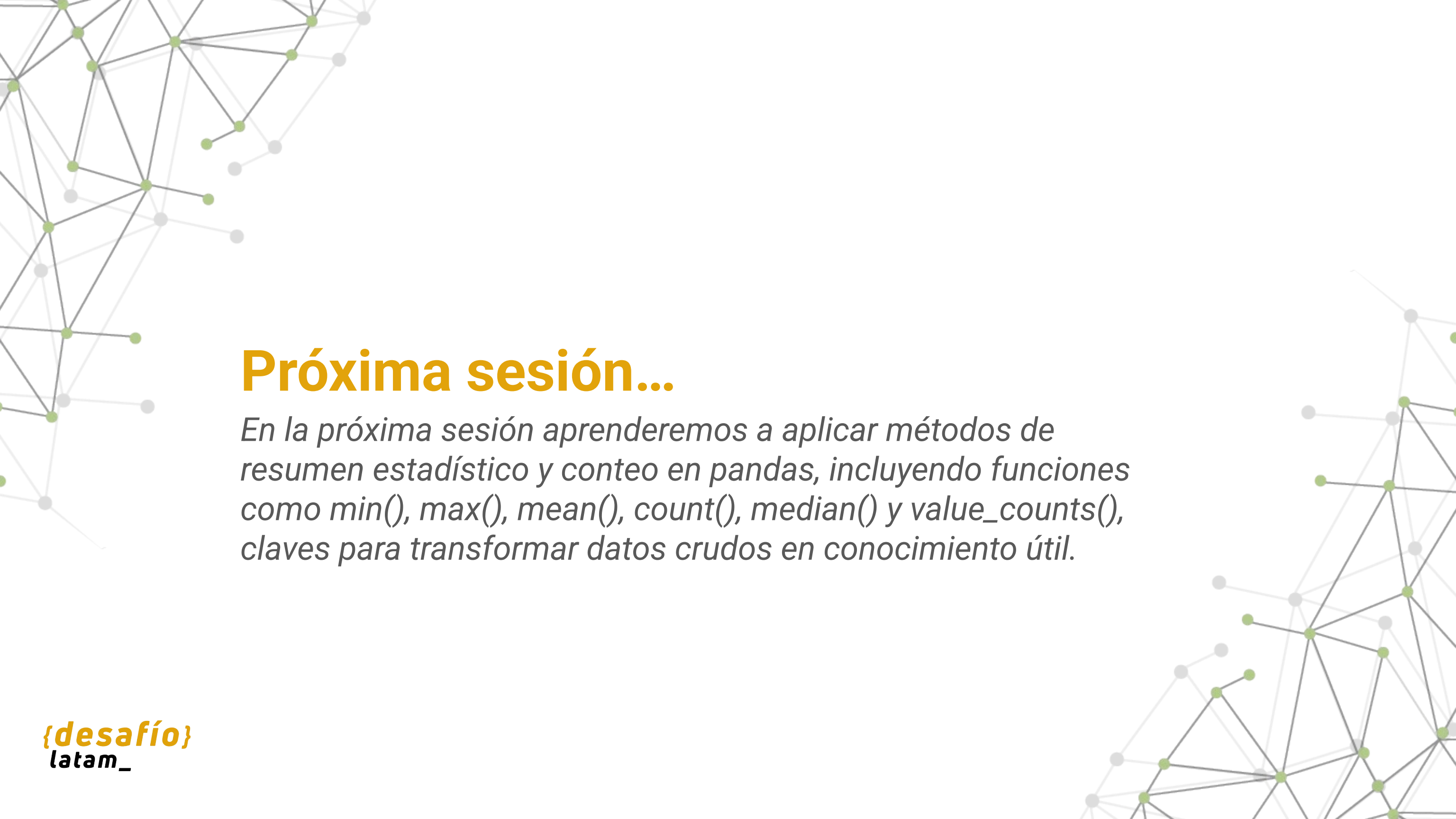


Interpretación

Comprendimos qué información nos entrega cada método y cómo interpretarla correctamente.

¿Por qué describe() no entrega valores para columnas de texto por defecto? ¿Qué método usarías para detectar si faltan datos?





Próxima sesión...

En la próxima sesión aprenderemos a aplicar métodos de resumen estadístico y conteo en pandas, incluyendo funciones como `min()`, `max()`, `mean()`, `count()`, `median()` y `value_counts()`, claves para transformar datos crudos en conocimiento útil.